Monte Carlo Pricing with Constant Parameters in QuantLib

Alexandre d'Hérissart - Raphael Lebel - Paul Trassaert - Thomas Lesage
IMT – Quantitative Finance

March 20, 2026

Abstract

Monte Carlo methods are widely used in quantitative finance to price derivatives when closed-form formulas are not available. In QuantLib, Monte Carlo pricing engines typically rely on time-dependent term structures for interest rates and volatility. However, repeatedly querying these structures during simulation introduces a significant computational cost.

In this project, we modify several Monte Carlo engines in QuantLib so that they can optionally use constant parameters extracted from the term structures. We analyze the impact of this approximation on both pricing accuracy and computational performance.

Contents

1 Introduction **3**

2 Theoretical Framework: Black-Scholes Model **4**
2.1 Modeling of the Underlying Asset Price . . . 4
2.2 Standard Black-Scholes Model . . . 4
2.3 Monte Carlo Simulation . . . 4
2.4 Constant-Parameter Approximation . . . 5

3 Implementation **6**
3.1 Class `ConstantBlackScholesProcess` . . . 6
3.2 Drift and Diffusion . . . 6
3.3 Simulation of the Underlying Evolution . . . 7
3.4 Extraction of Constant Parameters . . . 7
3.5 Code Factorization and Helper Design . . . 8
3.6 Integration into Monte Carlo Engines . . . 9

4 Modification of Monte Carlo Engines **9**
4.1 Addition of the `withConstantParameters` Option . . . 9
4.2 Modification of the Path Generator . . . 9
4.3 Application to the Different Engines . . . 10
4.4 Avoiding Code Duplication . . . 10
4.5 Design Benefits . . . 11

5 Numerical Results **11**
5.1 Validation of the Modified Engines . . . 12
5.2 Performance Improvement . . . 12
5.3 Impact on Pricing Accuracy . . . 13
5.4 Discussion . . . 14

6 Discussion and Possible Improvements **14**
6.1 Why the European Option Is Unaffected . . . 15
6.2 Impact on Path-Dependent Options . . . 15
6.3 Computational Interpretation . . . 15
6.4 Possible Improvements . . . 16
6.4.1 Piecewise Constant Parameters . . . 16
6.4.2 Adaptive Time Discretization . . . 16
6.4.3 Variance Reduction Techniques . . . 16
6.4.4 Parallelization . . . 16
6.5 Practical Implications . . . 16

7 Conclusion **17**

1 Introduction

Monte Carlo methods play a central role in quantitative finance for the pricing of complex derivative instruments. Unlike simpler products, for which analytical formulas exist (such as the Black-Scholes formula for European options), many derivatives depend on the entire trajectory of the underlying asset. This is notably the case for Asian options or lookback options, where the value of the product depends on the average or the maximum of the underlying price over the lifetime of the contract. In such situations, simulation methods become a natural and widely used tool.

The *QuantLib* library provides several pricing engines based on Monte Carlo simulations. These engines typically rely on a stochastic process of type `GeneralizedBlackScholesProcess`, which models the evolution of the underlying asset under the risk-neutral measure. In this framework, the model parameters — in particular the risk-free interest rate, the dividend yield, and the volatility — are not assumed to be constant, but are described by *term structures*. These structures allow for a flexible representation of the time dependence of these parameters based on market data.

Although this approach is both realistic and flexible, it introduces a non-negligible computational cost. During a Monte Carlo simulation, the stochastic process must be evaluated at each time step and for each simulated path. In this context, repeatedly querying the term structures to interpolate interest rates or volatilities can significantly slow down the execution of the simulations.

The objective of this project is to investigate to what extent this computational cost can be reduced by using an approximation in which the model parameters are assumed to be constant. More precisely, the idea is to extract constant values for the interest rate, the dividend yield, and the volatility from the term structures, typically at the option maturity, and to use these values throughout the simulation.

To implement this approach, we introduce a new stochastic process called `ConstantBlackScholesProcess`. This process corresponds to a simplified version of the Black-Scholes model in which the parameters are constant. This simplification notably allows for a more efficient simulation of the underlying asset dynamics.

We then modify several Monte Carlo pricing engines in QuantLib in order to allow them to use this simplified process when the corresponding option is activated. Three engines have been modified in this project:

- the Monte Carlo engine for European options,
- the Monte Carlo engine for discrete arithmetic Asian options,
- the Monte Carlo engine for fixed-strike lookback options.

The objective of this study is twofold. First, we verify that the results obtained with non-constant parameters are identical to those produced by the original QuantLib engines. Second, we analyze the impact of the constant-parameter approximation on both computation time and pricing accuracy.

The remainder of this report is organized as follows. Section 2 presents the theoretical framework of the Black-Scholes model used for the simulations. Section 3 describes the implementation of the `ConstantBlackScholesProcess`. Section 4 details the modifications made to the Monte Carlo engines. Section 5 presents the numerical results and compares the performance of the different approaches. Finally, Section 6 discusses the limitations of the approximation and proposes possible improvements.

2 Theoretical Framework: Black-Scholes Model

2.1 Modeling of the Underlying Asset Price

In the context of derivative pricing, it is common to model the evolution of a financial asset using a stochastic process. In the Black-Scholes framework, the price of the underlying asset S_t is assumed to follow a stochastic differential equation of the form

$$dS_t = (r(t) - q(t))S_t dt + \sigma(t, S_t)S_t dW_t,$$

where:

- S_t denotes the price of the underlying asset at time t,
- $r(t)$ is the risk-free interest rate,
- $q(t)$ is the continuous dividend yield,
- $\sigma(t, S_t)$ is the volatility,
- W_t is a standard Brownian motion.

In the QuantLib library, this dynamic is implemented through the `GeneralizedBlackScholesProcess` class. This process allows modeling the parameters r, q, and σ using term structures, enabling their dependence on time and, potentially, on the level of the underlying asset.

This formulation is particularly flexible, as it allows the direct use of market data such as yield curves and implied volatility surfaces.

2.2 Standard Black-Scholes Model

In the standard Black-Scholes model, the parameters are assumed to be constant. The stochastic differential equation then becomes

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t.$$

This equation describes a Geometric Brownian Motion. An important property of this process is that it admits an explicit analytical solution.

Integrating this equation over a time interval Δt, we obtain:

$$S_{t+\Delta t} = S_t \exp\left((r - q - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z\right),$$

where Z is a standard normally distributed random variable.

This formula plays a fundamental role in Monte Carlo methods, as it allows the direct simulation of the evolution of the underlying asset using Gaussian random draws.

2.3 Monte Carlo Simulation

In a Monte Carlo simulation, a large number of trajectories of the underlying asset are generated by applying the previous relation at each time step. For each simulated path, the option payoff is computed, and the average of these values provides an estimate of the option price under the risk-neutral measure.

Mathematically, the price of an option can be expressed as

$$V_0 = e^{-rT}\mathbb{E}[\,\text{Payoff}(S_t)\,],$$

where T denotes the maturity of the option.

For path-dependent derivatives, such as Asian or lookback options, the payoff depends on the entire trajectory of the underlying asset:

- for an Asian option, the payoff depends on the average of the simulated prices,
- for a lookback option, the payoff depends on the maximum or minimum value reached by the underlying asset.

It is precisely in these cases that Monte Carlo methods become particularly useful.

2.4 Constant-Parameter Approximation

In the standard QuantLib implementation, the parameters of the process are obtained from the term structures at each step of the simulation. This requires repeated interpolation in order to determine the instantaneous values of the interest rate and the volatility.

To reduce this computational cost, we consider in this project an approximation in which time-dependent parameters are replaced by constant values. More specifically, the constant parameters are extracted from the term structures at the option maturity:

$$r = r(T), \qquad q = q(T), \qquad \sigma = \sigma(T, S_0).$$

These values are then used throughout the simulation.

This approximation significantly simplifies the dynamics of the process and avoids repeated access to the term structures. It leads to the introduction of a new stochastic process, called `ConstantBlackScholesProcess`, which corresponds to a simplified version of the Black-Scholes model with constant parameters.

In the following sections, we describe the implementation of this process and the modifications made to the Monte Carlo engines in order to take advantage of this approximation.

3 Implementation

In order to introduce the constant-parameter approximation into the Monte Carlo simulation engines, we implemented a new stochastic process called `ConstantBlackScholesProcess`. This process corresponds to a simplified version of the Black-Scholes model in which the interest rate, the dividend yield, and the volatility are assumed to be constant.

The objective of this implementation is to reproduce the dynamics of the geometric Brownian motion while avoiding repeated calls to the term structures during the simulation.

3.1 Class `ConstantBlackScholesProcess`

We introduced a new class derived from `StochasticProcess1D`. This class takes as input the constant model parameters as well as the initial value of the underlying asset.

```
class ConstantBlackScholesProcess : public StochasticProcess1D {
  public:

    ConstantBlackScholesProcess(
        Real s0,
        Rate riskFreeRate,
        Rate dividendYield,
        Volatility volatility);

    Real x0() const override;
    Real drift(Time t, Real x) const override;
    Real diffusion(Time t, Real x) const override;

    Real evolve(Time t0, Real x0, Time dt, Real dw) const override;

  private:

    Real s0_;
    Rate r_;
    Rate q_;
    Volatility sigma_;
};
```

This class stores the constant parameters of the model:

- s_0: initial value of the underlying asset,
- r: risk-free interest rate,
- q: dividend yield,
- σ: constant volatility.

3.2 Drift and Diffusion

In the Black-Scholes model, the dynamics of the underlying asset are given by

$$dS_t = (r - q)S_t dt + \sigma S_t dW_t.$$

The `drift` and `diffusion` functions implement the drift and diffusion terms of this equation, respectively.

```
Real ConstantBlackScholesProcess::drift(Time, Real x) const {
    return (r_ - q_) * x;
}

Real ConstantBlackScholesProcess::diffusion(Time, Real x) const {
    return sigma_ * x;
}
```

The drift therefore corresponds to the term $(r-q)S_t$, while the diffusion corresponds to the term σS_t.

3.3 Simulation of the Underlying Evolution

The evolution of the process is implemented in the `evolve` method. Instead of using an Euler discretization, we rely on the analytical solution of the geometric Brownian motion:

$$S_{t+\Delta t} = S_t \exp\left((r - q - \frac{1}{2}\sigma^2)\Delta t + \sigma\sqrt{\Delta t}Z\right),$$

where Z follows a standard normal distribution.

This formula allows us to directly simulate the evolution of the underlying asset between two time points.

```
Real ConstantBlackScholesProcess::evolve(
    Time, Real x0, Time dt, Real dw) const {

    Real driftTerm =
        (r_ - q_ - 0.5 * sigma_ * sigma_) * dt;

    Real diffusionTerm =
        sigma_ * std::sqrt(dt) * dw;

    return x0 * std::exp(driftTerm + diffusionTerm);
}
```

This implementation exactly matches the analytical solution of the geometric Brownian motion.

3.4 Extraction of Constant Parameters

The constant parameters are extracted from the term structures of the Black-Scholes process used in QuantLib.

More precisely, we extract the values of the interest rate, the dividend yield, and the volatility at the option maturity.

For the volatility, the extraction is performed using the option strike, consistently with the standard Black volatility parametrization $\sigma(T, K)$, where T is the maturity and K the strike of the option.

It should be noted that the choice of extracting the parameters at maturity is a modeling simplification. Alternative approaches could be considered, such as using an average volatility over the time horizon or a forward-looking implied volatility. This choice may slightly impact the pricing accuracy, especially for path-dependent products.

In our numerical experiments, the volatility term structure is given by a `BlackVarianceCurve`, which depends only on time. As a result, using the strike instead of the spot does not materially affect the results, although it remains the correct modeling choice in the general case.

```
Rate r =
process->riskFreeRate()
       ->zeroRate(T, dayCounter,
                  Continuous, NoFrequency)
       .rate();

Rate q =
process->dividendYield()
       ->zeroRate(T, dayCounter,
                  Continuous, NoFrequency)
       .rate();

Volatility sigma =
process->blackVolatility()
       ->blackVol(T, strike);
```

These parameters are then used to construct the simplified process:

```
auto constantProcess =
    ext::make_shared<ConstantBlackScholesProcess>(
        s0, r, q, sigma);
```

3.5 Code Factorization and Helper Design

In order to avoid code duplication across the different Monte Carlo engines, we introduced a helper component responsible for extracting the constant parameters and building the associated stochastic process.

Instead of duplicating the parameter extraction logic in each engine, a dedicated helper function is used to:

- extract (s_0, r, q, σ) from the original `GeneralizedBlackScholesProcess`,
- construct a `ConstantBlackScholesProcess`,
- return it in a unified and reusable way.

This design significantly improves:

- **code maintainability**, by centralizing the logic in a single location,
- **consistency**, ensuring that all engines use exactly the same parameter extraction methodology,
- **extensibility**, making it easier to modify the approximation method in the future.

This choice directly addresses the project requirement of avoiding duplication across the modified engines.

3.6 Integration into Monte Carlo Engines

The Monte Carlo engines were modified to allow the optional use of this simplified process.

A new option was added to the engines:

```
.withConstantParameters()
```

When this option is activated, the engine uses `ConstantBlackScholesProcess` to generate the simulated paths. Otherwise, it continues to use the standard QuantLib process.

Thanks to the helper abstraction, the integration remains lightweight in each engine, as the construction of the constant process is delegated to a shared utility rather than reimplemented multiple times.

4 Modification of Monte Carlo Engines

Following the introduction of the `ConstantBlackScholesProcess`, the next step is to modify the Monte Carlo simulation engines so that they can optionally use this simplified process.

Three pricing engines have been modified in this project:

- the Monte Carlo engine for European options;
- the Monte Carlo engine for discrete arithmetic Asian options;
- the Monte Carlo engine for fixed-strike lookback options.

The objective of these modifications is to allow the optional use of constant parameters while preserving the original behavior of the engines when this option is not activated.

4.1 Addition of the `withConstantParameters` Option

A new option has been added to the Monte Carlo engine builders:

```
.withConstantParameters()
```

This option allows the user to choose between two simulation modes:

- the standard QuantLib process with time-dependent parameters;
- the `ConstantBlackScholesProcess` with constant parameters.

When this option is activated, the constant parameters are extracted from the term structures and used to construct a new simplified stochastic process.

4.2 Modification of the Path Generator

The main change occurs in the generation of the underlying asset paths.

In the original implementation, paths are generated using the time-dependent Black-Scholes process:

```
return ext::shared_ptr<path_generator_type>(
    new path_generator_type(process_,
                            grid,
                            gen,
                            brownianBridge_));
```

This process relies on the term structures to obtain interest rates and volatilities at each step of the simulation.

In the modified version, when the `withConstantParameters()` option is enabled, the construction of the constant process is delegated to a shared helper function:

```
auto constantProcess =
    makeConstantBlackScholesProcess(process, T);
```

The path generator then uses this new process:

```
return ext::shared_ptr<path_generator_type>(
    new path_generator_type(constantProcess,
                            grid,
                            gen,
                            brownianBridge_));
```

This modification removes the dependency on repeated term-structure evaluations during the simulation loop, which significantly improves performance.

4.3 Application to the Different Engines

The same approach has been applied to the three simulation engines studied:

- `MCEuropeanEngine_2`
- `MCDiscreteArithmeticASEngine_2`
- `MCFixedLookbackEngine_2`

In each engine, the main modification consists in introducing a conditional mechanism that selects between the original process and the constant-parameter process when generating the simulation paths.

4.4 Avoiding Code Duplication

A key design objective of this implementation was to avoid code duplication across the three engines.

To achieve this, the logic responsible for:

- extracting the constant parameters from the term structures,
- constructing the `ConstantBlackScholesProcess`,

has been fully factorized into a shared helper component.

As a result:

- the three engines rely on a single, centralized implementation of the constant-parameter extraction,
- the code remains consistent across all engines,
- any future modification of the approximation method can be implemented in one place only.

This design directly addresses the project requirement of avoiding duplication while improving maintainability and code clarity.

4.5 Design Benefits

This modular approach provides several advantages:

- **Performance improvement**: removing repeated calls to term structures significantly reduces computation time;
- **Code maintainability**: shared logic avoids inconsistencies between engines;
- **Extensibility**: alternative parameter extraction strategies can be implemented easily;
- **Backward compatibility**: when the option is disabled, the engines behave exactly like the original QuantLib implementations.

In the next section, we analyze the numerical results obtained under the different configurations and compare the performance in terms of computation time and pricing accuracy.

5 Numerical Results

In this section, we compare the results obtained using three different Monte Carlo engines:

- the original QuantLib implementation (*Old Engine*),
- the modified implementation using the original time–dependent parameters (*Non-Constant*),
- the modified implementation using the constant-parameter approximation (*Constant*).

All simulations were performed with the following parameters:

- Number of time steps: 10
- Number of Monte Carlo samples: 10^6
- Random seed: 42

The numerical results are summarized in Table 1.

	Old Engine		Non-Constant		Constant	
Option	NPV	Time (s)	NPV	Time (s)	NPV	Time (s)
European	4.17073	1.32851	4.17073	1.31197	4.17073	0.16802
Asian	0.72943	1.18694	0.72943	1.20794	0.73117	0.14339
Lookback	5.99980	1.30011	5.99980	1.32523	5.99705	0.14563

Table 1: Comparison of option prices and computation times for the different Monte Carlo engines

5.1 Validation of the Modified Engines

The first observation concerns the comparison between the original QuantLib engine and the modified engine using non-constant parameters.

The results show that the computed prices are identical for all three option types. This confirms that the modifications introduced in the Monte Carlo engines do not alter the pricing algorithm when the constant-parameter approximation is not activated.

This result validates the correctness of the implementation.

5.2 Performance Improvement

The main objective of the constant-parameter approximation is to reduce the computational cost of Monte Carlo simulations.

The results show a significant reduction in execution time. For all three option types, the computation time decreases from approximately 1.2–1.3 seconds to roughly 0.14–0.15 seconds.

Figure 1 illustrates the execution times for the three implementations.

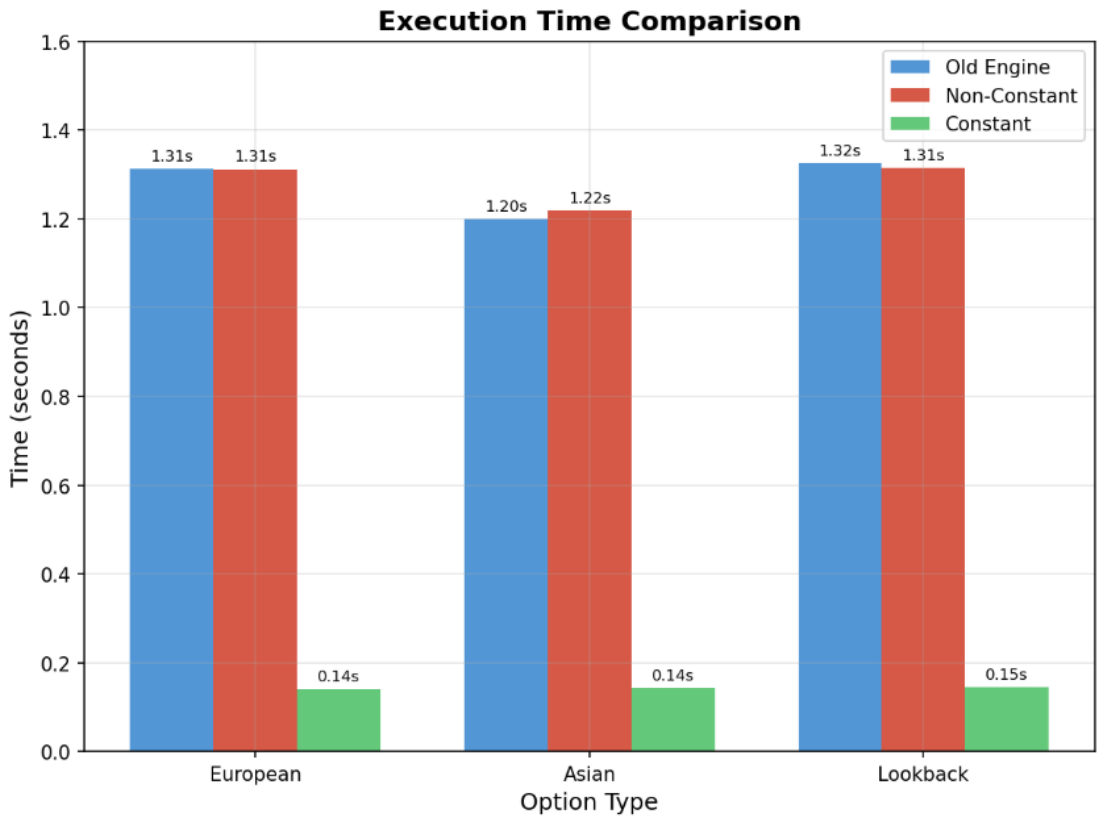

Figure 1: Comparison of execution times for the three engines

To quantify the performance improvement, we define the speedup factor:

$$\text{Speedup} = \frac{\text{Time}_{\text{original}}}{\text{Time}_{\text{constant}}}$$

The resulting acceleration factors are presented in Figure 2.

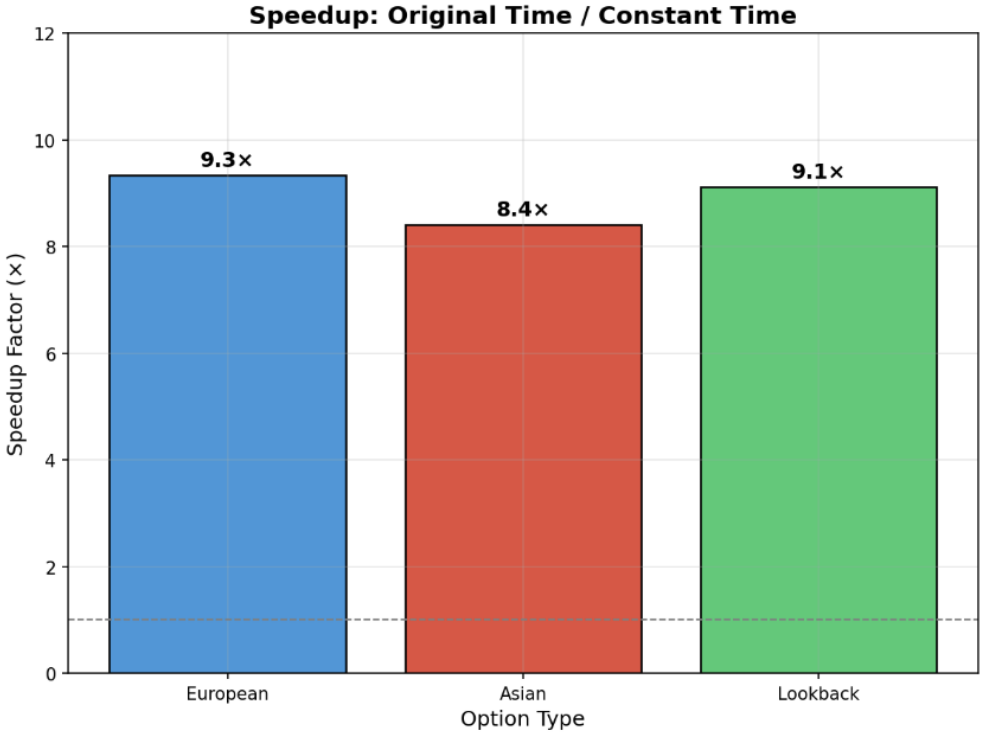

Figure 2: Speedup obtained with the constant-parameter approximation

The speedup factors obtained are approximately:

- European option: 9.3×
- Asian option: 8.4×
- Lookback option: 9.1×

This improvement is explained by the fact that the original implementation repeatedly queries the term structures (interest rate and volatility) during the simulation. These operations involve interpolation and are relatively expensive.

In contrast, the constant-parameter approximation extracts these parameters once before the simulation begins, which significantly reduces the computational overhead during path generation.

5.3 Impact on Pricing Accuracy

Using constant parameters introduces an approximation in the underlying stochastic process. Therefore, it is important to evaluate the impact of this simplification on option prices.

For the European option, the computed price remains identical across all implementations. This result is expected because the payoff of a European option depends only on the terminal value S_T, and the constant-parameter approximation provides a very accurate representation of the process over the considered time horizon.

For path-dependent options, small differences appear. The Asian option price changes from 0.72943 to 0.73117, while the lookback option price changes from 5.99980 to 5.99705.

To quantify this effect, we compute the relative pricing error:

$$\text{Relative Error} = \frac{|V_{\text{constant}} - V_{\text{original}}|}{V_{\text{original}}}$$

The results are illustrated in Figure 3.

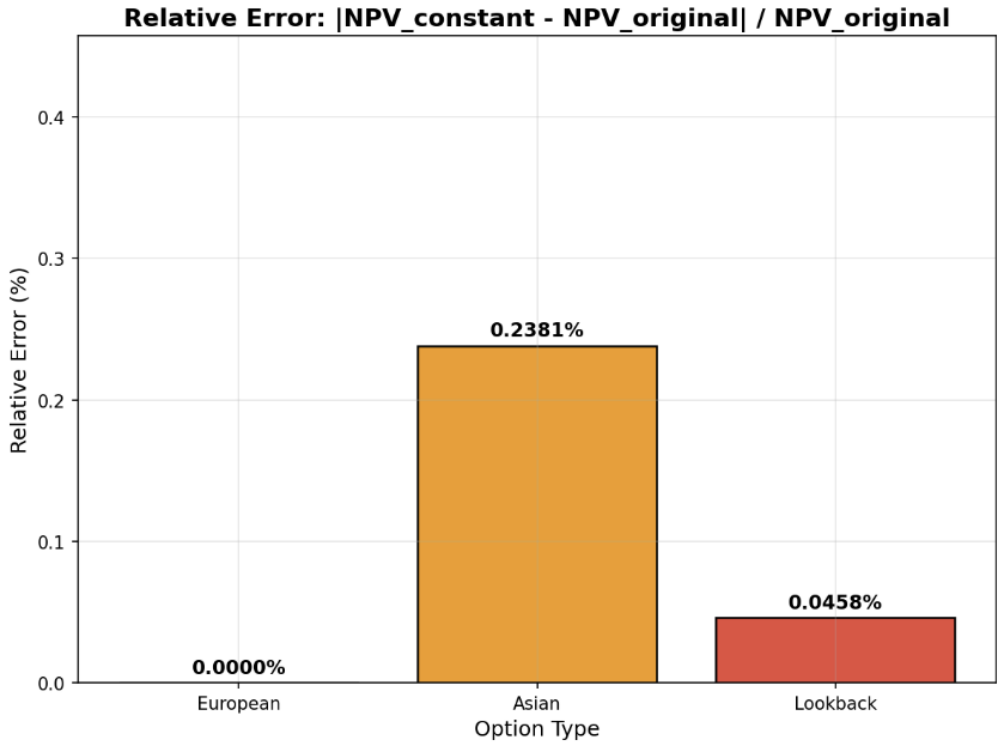

Figure 3: Relative pricing error introduced by the constant-parameter approximation

The relative errors remain extremely small:

- European option: 0.0000%
- Asian option: 0.2381%
- Lookback option: 0.0458%

These results indicate that the constant-parameter approximation preserves pricing accuracy while providing a substantial computational gain.

5.4 Discussion

The results highlight an important trade-off between computational efficiency and model precision.

While the constant-parameter approximation slightly alters the dynamics of the underlying process, the impact on option prices remains negligible in the scenarios considered. At the same time, the reduction in computational cost is significant, with speedups close to an order of magnitude.

Such improvements can be particularly valuable in practical applications where large numbers of Monte Carlo simulations are required, such as calibration procedures, risk management, or portfolio valuation.

6 Discussion and Possible Improvements

The numerical results presented in the previous section highlight the main benefit of the constant-parameter approximation: a substantial reduction in computational time with only a minimal impact on pricing accuracy.

In this section, we analyze these results in more detail and discuss possible improvements and extensions of the approach.

6.1 Why the European Option Is Unaffected

One of the most striking results is that the European option price remains identical across all implementations.

This behavior can be explained by the nature of the European payoff. The payoff of a European option depends only on the terminal value of the underlying asset S_T:

$$\text{Payoff} = f(S_T).$$

As a result, the Monte Carlo simulation only needs to correctly reproduce the distribution of S_T.

In the Black–Scholes framework, replacing time-dependent parameters with appropriately chosen constant parameters (in particular evaluated at maturity) preserves a very close approximation of the terminal distribution. Therefore, the resulting option price remains essentially unchanged.

This explains why the European option exhibits zero relative error in our numerical experiments.

6.2 Impact on Path-Dependent Options

The situation is slightly different for path-dependent options such as Asian and lookback options.

Unlike European options, these derivatives depend on the entire trajectory of the underlying asset:

- Asian options depend on the average of the asset price along the path,
- Lookback options depend on the maximum or minimum value reached during the path.

When the time-dependent parameters of the Black–Scholes process are replaced by constant values, the dynamics of the simulated paths are slightly modified.

While the overall distribution of the terminal value remains close to the original one, the intermediate path behavior can differ. This explains why small pricing differences appear for these derivatives.

However, the relative errors observed in our experiments remain very small (well below 1%), which indicates that the constant-parameter approximation still provides a very good representation of the process in this setting.

6.3 Computational Interpretation

From a computational perspective, the main cost of the original implementation comes from repeated queries to the term structures (interest rate curve and volatility surface).

At each time step of the simulation, the engine must:

- compute the instantaneous risk-free rate,
- compute the instantaneous volatility,
- interpolate these values from the term structures.

These operations are relatively expensive because they involve interpolation and access to complex data structures.

By extracting constant parameters before the simulation begins, these repeated operations are eliminated. The Monte Carlo loop then becomes significantly lighter, which explains the large speedup observed in the experiments.

In our numerical results, this leads to a reduction of the computation time by nearly an order of magnitude, confirming the practical relevance of this optimization.

6.4 Possible Improvements

Although the constant-parameter approximation already provides significant performance gains, several improvements could be considered.

6.4.1 Piecewise Constant Parameters

Instead of using a single constant volatility and interest rate for the entire simulation horizon, one could use a piecewise constant approximation.

For example, the simulation period could be divided into several time intervals, each with its own constant parameters extracted from the term structures.

This approach would preserve most of the computational benefits while providing a more accurate approximation of the original model dynamics.

6.4.2 Adaptive Time Discretization

Another possible improvement would consist in adapting the time discretization to the structure of the payoff.

For instance, Asian options depend on a finite set of fixing dates. The time grid used in the Monte Carlo simulation could therefore be aligned with these dates, reducing the number of unnecessary intermediate simulation steps.

Such an approach could further reduce computational costs without sacrificing accuracy.

6.4.3 Variance Reduction Techniques

The performance of Monte Carlo simulations can also be improved through variance reduction techniques.

Examples include:

- antithetic variates,
- control variates,
- quasi-Monte Carlo sequences.

These methods reduce the number of simulation paths required to achieve a given level of accuracy, which directly reduces computation time.

6.4.4 Parallelization

Monte Carlo simulations are inherently parallelizable since individual paths are independent.

Modern computing architectures such as multi-core CPUs or GPUs could therefore be exploited to accelerate the simulations even further. Parallel Monte Carlo implementations are widely used in practice for large-scale financial computations.

6.5 Practical Implications

The results obtained in this project illustrate an important principle in quantitative finance: a well-designed approximation can significantly reduce computational costs while preserving pricing accuracy.

In addition, the use of a shared helper to centralize the construction of the constant-parameter process ensures that this approximation is implemented consistently across different pricing engines, improving both robustness and maintainability.

In practical applications such as risk management, portfolio valuation, or calibration procedures, where thousands of option prices must be computed repeatedly, such performance improvements can be extremely valuable.

The constant-parameter approximation presented in this project therefore represents a simple but effective technique to accelerate Monte Carlo pricing engines while maintaining reliable results.

7 Conclusion

In this project, we investigated the impact of replacing time-dependent model parameters by constant approximations within Monte Carlo pricing engines in QuantLib.

We introduced a new stochastic process, `ConstantBlackScholesProcess`, designed to replicate the dynamics of the Black-Scholes model under constant parameters. This process was then integrated into three Monte Carlo engines for European, Asian, and Lookback options through an additional optional parameter.

In addition, particular attention was paid to the design of the implementation, notably by factorizing the extraction of constant parameters into a shared helper component. This allowed us to avoid code duplication across the different engines while ensuring consistency and maintainability.

The numerical results confirm that:

- the modified engines with non-constant parameters reproduce exactly the results of the original QuantLib engines;
- the constant-parameter approximation leads to a significant reduction in computation time, with speedups close to one order of magnitude;
- the pricing errors introduced by the approximation remain very small, especially for European options, and acceptable for path-dependent products.

These results highlight a clear trade-off between computational efficiency and modeling accuracy. In many practical applications, especially when fast pricing is required, the constant-parameter approximation provides an attractive alternative to the full term-structure-based model.

More broadly, this work illustrates the importance of both modeling choices and software design in quantitative finance: even simple approximations, when carefully implemented, can lead to substantial performance gains while preserving reliable results.